Lab 7: Fuzzing for Input Validation Bugs

Student details

Field	Details
Name	Omkar
SRN	PES1UG24CS310
Lab	Lab 7: Fuzzing for Input Validation Bugs
Assigned repo	SETAPESU26/L6_L7_05_blood_donation_screening
Submission repo	PES1UG24CS310_SE_LAB_SUBMISSIONS, folder "Lab 7: Fuzzing for Input Validation Bugs"

Step 1: Installing dependencies

```
pip install hypothesis
pip install pytest
```

Step 2: Running the buggy processor.py

```
python processor.py
```

```
@omkar040806 ➜/workspaces/PES1UG24CS310_SE_LAB_SUBMISSIONS/Lab 7: Fuzzing for Input Validation Bugs (main) $ python processor.py
==== Running validate_blood_type ====
Enter blood type (e.g. A+, O-, AB+): A+
Validated: A Positive

==== Running calculate_pulse_pressure ====
Enter BP (e.g. 120/80): 120/80
BP Metrics: Diff: 40, Ratio: 1.5

==== Running parse_donor_registry ====
Enter record (ID|Weight|Hemoglobin): D101|65|13.5
Status: Donor D101: Eligible
@omkar040806 ➜/workspaces/PES1UG24CS310_SE_LAB_SUBMISSIONS/Lab 7: Fuzzing for Input Validation Bugs (main) $ python processor.py
==== Running validate_blood_type ====
Enter blood type (e.g. A+, O-, AB+): XYZ
Traceback (most recent call last):
  File "/workspaces/PES1UG24CS310_SE_LAB_SUBMISSIONS/Lab 7: Fuzzing for Input Validation Bugs/processor.py", line 50, in <module>
    main()
    ~~~~^^
  File "/workspaces/PES1UG24CS310_SE_LAB_SUBMISSIONS/Lab 7: Fuzzing for Input Validation Bugs/processor.py", line 39, in main
    print("Validated:", validate_blood_type(b_input))
                        ~~~~~~~~~~~~~~~~~~~~^^^^^^^^
  File "/workspaces/PES1UG24CS310_SE_LAB_SUBMISSIONS/Lab 7: Fuzzing for Input Validation Bugs/processor.py", line 10, in validate_blood_type
    return f"{antigen} {rh_map[rh_char]}"
                        ~~~~~~^^^^^^^^^
KeyError: 'Z'
```

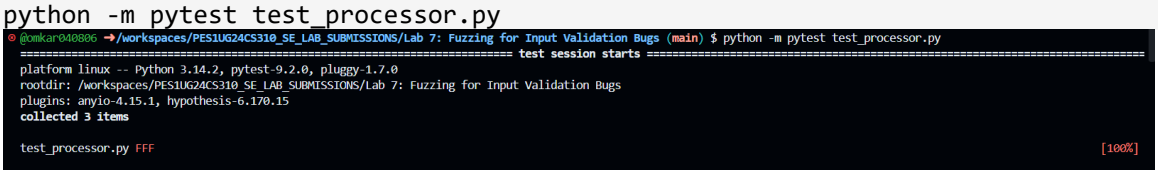
Screenshot 1: buggy processor.py, a valid run followed by the KeyError crash

Step 3: Completing test_processor.py with Hypothesis

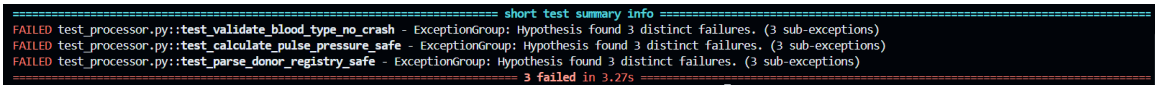
Each test was completed with try/except, so Hypothesis (st.text() | st.none()) reports any input that crashes the function. The completed code is submitted as test_processor.py.

Step 4: Running the tests on the buggy code

```
python -m pytest test_processor.py
```



Screenshot 2: pytest collects 3 items, and all three fail (FFF)

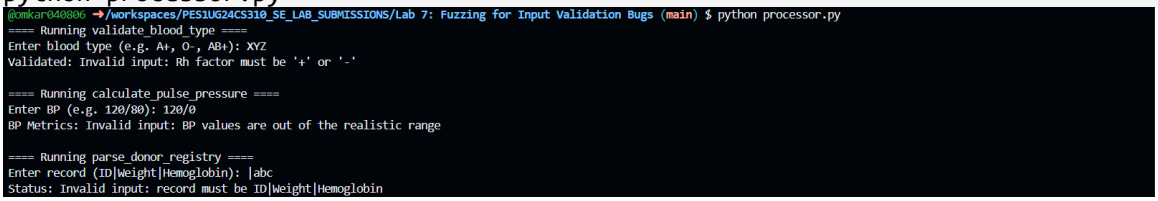


Screenshot 3: the short test summary shows 3 failed

Step 5: Running the fixed processor.py

processor.py was hardened against the inputs found in Step 4 and run again. The fixed script is submitted as processor.py.

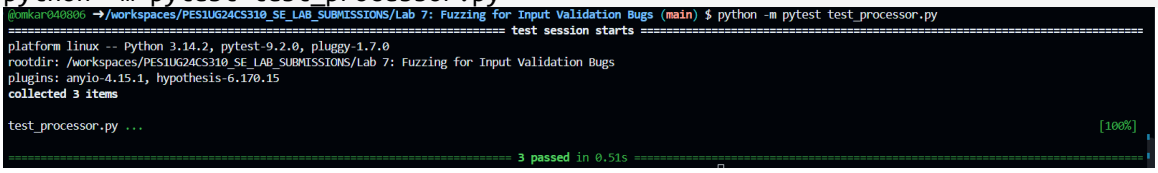
```
python processor.py
```



Screenshot 4: fixed processor.py handling invalid input without crashing

Step 6: Re-running the tests on the fixed code

```
python -m pytest test_processor.py
```



Screenshot 5: 3 passed